

Administration Linux

PRINCIPAUX RÉPERTOIRES SYSTÈMES

- Répertoires standards :

/ Répertoire racine (ou root) contenant tous les répertoires.

/home Répertoire contenant les répertoires personnels de tous les utilisateurs autres que root (peut être changé)

/root Répertoire personnel de l'administrateur système root.

- **Répertoires système :**

/bin Répertoire contenant les commandes et utilitaires employés par tous les utilisateurs (ls, rm, cp, etc..)

/boot Répertoire contenant des informations permettant le chargement de Linux.

/dev Répertoire contenant tous les fichiers périphériques permettant d'accéder aux composants matériels.

/etc Répertoire contenant les commandes et fichiers de configuration nécessaires à l'administration système.

/lib Répertoire contenant les bibliothèques communes à tous les utilisateurs

/sbin Répertoire contenant les commandes et utilitaires utilisées seulement root

/tmp Répertoire contenant les fichiers temporaires.

/usr Répertoire composé d'un certain nombre de sous répertoires utilisés par l'ensemble des utilisateurs.

/var Répertoire spécial utilisé par le système pour stocker des données souvent modifiées, spoule d'impression, les journaux système

GESTION DES UTILISATEURS

GESTION DES UTILISATEURS

Un compte utilisateur :

- définition d'un environnement utilisateur
- définition de son identification pour le système

Opérations possibles :

- création
- modifications
- destruction

- **Le fichier /etc/passwd**

Il contient les informations sur les identifications des utilisateurs.

Exemple :

```
root :x:0:0:root:/root:/bin/bash
```

```
kmaster:x:500:500:kmaster:/home/kmaster:/bin/bash
```

Nota le deuxième champ (x) spécifie q'il y a un mot de passe et que celui-ci est crypté (dans /etc/shadow).

Format du fichier /etc/passwd

Le format est composé de 7 champs séparés par le caractère ::

- champ 1 : **login**
Nom sous lequel un ordinateur connaît un individu.
8 caractères au plus. Pas de majuscules. Eviter les caractères accentués.
- champ 2 : **mot de passe**
Mot de passe de l'utilisateur stocké **sous forme chiffrée non déchiffrable.**

Au login :

```
strcmp("saeLydiaFuF5o", crypt("sa", "*****"))
```

Nécessité d'éduquer les utilisateurs pour choisir un bon mot de passe.

Les logiciels de crack de mots de passe ne déchiffrant pas les mots de passe , ils font des essais à base de dictionnaires.

- champ 3 : **UID**
Identificateur numérique compris entre 0 et 32767 (32 bit).
Cette valeur doit être unique au sein des utilisateurs
- champ 4 : **GID**
Identificateur numérique compris entre 0 et 32767 (32 bit).
Cette valeur doit être unique au sein des groupes.
- champ 5 : **gecos**
Identité en clair de l'utilisateur ou commentaire
- champ 6 : **homedir**
Répertoire par défaut.
- champ 7 : **shell**
Shell par défaut.

- Quelques conseils :
 - Il faut toujours trier les lignes de `/etc/passwd` selon l'ordre numérique des UID
 - Il ne faut jamais laisser de comptes sans mot de passe.

Le fichier /etc/shadow

L'idée part de la constatation que, si beaucoup de programmes accèdent au contenu de /etc/passwd pour les informations concernant UID, homedir, shell, peu en revanche y accèdent pour le mot de passe.

On supprime donc le mot de passe chiffré du fichier pour le stocker dans un fichier à accès plus restreints.

-rw-r--r-- root root 10557 Sep 15 22:51 /etc/passwd

-rw----- root root 13318 Sep 15 22:50 /etc/shadow

L'ancien mot de passe chiffré est remplacé par exemple par un caractère comme * ou x :

ensa:x:4332:1000::/users/adm/ensa:/bin/bash

- Plusieurs formats de fichiers shadow sont utilisés par les constructeurs qui ne se sont pas mis d'accord.

Le fichier /etc/group

Ce fichier permet de faire le lien entre les numéros de groupe et leurs noms.

Le format est composé de 4 champs séparés par :

Nom de groupe:Champ spécial:Numéro de groupe:Membre1,Membre2, ...

champ 4 : **membres**

Liste de noms de login séparés par des **virgules**.

Ajout d'un utilisateur

Selon le système, elle est automatisée sous la forme d'une commande d'administration :

Système	Programme
----------------	------------------

Aix	smit
Solaris	admintool
HP	sam
Linux	useradd

Sous Linux:

- **useradd** : (utiliser le man)

```
$ useradd toto .....
```

Puis initialiser le mot de passe

```
$ passwd toto
```

Actions schématiques à accomplir lors de la création d'un compte :

1. choix de l'UID et du GID en fonction du service d'appartenance de la personne
2. choix du homedir (en fonction du service ?)
3. choix du nom de login selon la politique locale
4. choix du shell de login
5. ajout de l'utilisateur dans /etc/group
6. création du homedir
7. copie des fichiers de configuration de l'environnement (.profile, .cshrc,.xsession etc.)
8. attribuer le homedir créé à l'utilisateur par **chown + chgrp**
9. initialisation du mot de passe

- **Compte root**

Sa particularité vient de son **UID == 0**.

Quelques règles de sécurité :

1. L'utilisateur root n'a pas **.** dans son PATH (précédence de la commande locale par rapport à la commande système).
2. L'utilisateur root a 022 pour umask (accessibilité indispensable de certains fichiers par les utilisateurs normaux).
3. Eviter d'avoir / comme homedir pour root (pollution de / par les fichiers de configuration en

- Suppression d'un utilisateur avec la commande userdel :
\$ userdel -r toto

L'option -r permet de supprimer le répertoire personnel de l'utilisateur à supprimer.

- Ajout d'un groupe avec la commande groupadd :
\$ groupadd ftpusers
- Suppression d'un groupe avec la commande groupdel :
\$ groupdel ftpusers

Changement de mots de passe

La commande pour changer de mot de passe d'un utilisateur dans /etc/passwd ou /etc/shadow est "**passwd**" :

```
# passwd ensa
```

```
Changing password for ensa
```

```
New password: XXXXXXXXX
```

```
Retype new password: XXXXXXXXX
```

root n'a jamais besoin de connaître le mot de passe d'un utilisateur pour le changer.

Commande su

La commande su permet de changer d'identité :

su [-] utilisateur2

Pour hériter complètement de l'identité, utiliser l'option "-" de la cmd "su"

Par sécurité:

Utiliser /bin/su

Fichiers de configuration des shells

<code>/etc/profile</code>	<code>/bin/sh</code>	Fichier global de configuration de sh. Exécuté lors d'une connexion.
<code>\$HOME/.profile</code>	<code>/bin/sh</code>	Fichier exécuté lors d'une connexion.
<code>\$HOME/.forward</code>	courrier électronique	Redirection du courrier si nécessaire.
<code>\$HOME/.mailrc</code>	courrier électronique	Configuration d'options, d'alias. . .
<code>\$HOME/.exrc</code>	vi/ed	Configuration des éditeurs de texte vi/ed.
<code>\$HOME/.xsession</code> <code>\$HOME/.xinitrc</code> <code>\$HOME/.Xresources</code> <code>\$HOME/.Xdefaults</code>	système de multifenêtrage	

Tâches périodiques : cron

Utilitaire cron

Imaginez que vous souhaitez lancer une tâche durant la nuit, ou bien effectuer des sauvegardes régulières de vos données, remettre à jour la base de vos packages...

L'objectif de ce paragraphe est de vous présenter le fonctionnement et le paramétrage l'outil de programmation sous Linux (**cron**).

Il existe un système automatisant ces lancement : **cron**

Ce mécanisme n'est pas restreint à root a priori.

Lancement du démon cron

Le démon cron est lancé au moment du boot :

Sur Linux :

```
% ls -l /etc/rc.d/init.d/crond
```

```
-rwxr-xr-x 1 root root 1031 Feb 3 2000 /etc/rc.d/init.d/crond
```

Sur Solaris :

```
% ls -l /etc/init.d/crond
```

```
-rwxr--r-- 4 root sys 513 Jul 16 1997 /etc/init.d/cron
```

Fichiers crontab

Les actions à lancer périodiquement sont indiquées dans un fichier texte au format :

minutes heures jour-du-mois mois jour-de-semaine commande

champ 1 : minutes, de 00 à 59.

champ 2 : heures, de 00 à 23.

champ 3 : jour du mois, de 1 à 31.

champ 4 : mois, de 1 à 12.

champ 5 : jour de la semaine, de 0 (dimanche) à 6 (samedi).

champ 6 : commande à exécuter.

Par exemple :

```
#####
```

```
15 * * * * /etc.local/cron/scripts/ntpdate
```

```
#####
```

```
00 21 * * 1 /etc.local/cron/scripts/tartare
```

```
00 21 * * 5 /etc.local/cron/scripts/tartare
```

```
#####
```

- L'emplacement des fichiers crontab dépend des systèmes :

Système	Program
AIX	<code>/var/spool/cron/crontabs/</code>
HP-UX	<code>/var/spool/cron/crontabs/</code>
FreeBSD	<code>/var/cron/tabs/</code>
Linux	<code>/var/spool/cron/</code>
Solaris	<code>/var/spool/cron/crontabs/</code>
etc.	

- Il y a un fichier crontab par utilisateur dans les directories mentionnés ci-dessus.

Exécution des fichiers crontab

Attention :

Les commandes s'exécutent sans terminal associé. Il n'y a donc pas de stdin ou stdout associés. Il faut réaliser soi-même les redirections requises.

Edition des fichiers crontab

Les fichiers crontab ont beau être au format texte, il ne faut pas les éditer manuellement. Il faut passer par l'intermédiaire de la commande crontab.

Il y a deux façons de faire selon les versions de la commande :

1. `crontab -e`

Cela lance l'éditeur de texte indiqué par la variable d'environnement EDITOR ou bien vi par défaut.

2. `crontab -l > myfile`

`vi myfile`

`crontab myfile`

Autorisation à utiliser cron

L'utilisation du mécanisme cron peut être contrôlé via deux fichiers `cron.allow` et `cron.deny`.

L'emplacement de ces fichiers dépend de chaque système

Linux: `/etc/cron/cron.allow`

Solaris : `/etc/cron.d/cron.allow`
`/etc/cron.d/cron.deny`

Fonctionnement du contrôle :

<code>cron.allow</code>	<code>cron.deny</code>	Utilisateurs autorisés
Présent	Présent	Ceux explicitement dans <code>cron.allow</code>
Présent	–	Ceux explicitement dans <code>cron.allow</code>
–	Présent	Tous sauf ceux dans <code>cron.deny</code>
–	–	Uniquement root

Il est conseillé de ne laisser l'accès à cron qu'à l'utilisateur root.

Résumé:

1)Le démon crond

2)Les fichiers « cron tables »

3)La commande crontab :

Gestion des processus

Pour chaque processus exécuté est stocké un certain nombre d'informations.

Ces données sont notamment conservées dans la table des processus, qui regroupe toutes les données d'identification essentielles du processus :

- Numéro de processus spécifique : PID;
- Numéro de processus parent (PPID), à partir duquel ce processus a été lancé ;
- Numéro d'utilisateur (*user id*, UID) ;
- Numéro de groupe (*group id*, GID) ;
- Durée du traitement utilisée (temps CPU) et priorité du processus ;
- Référence au répertoire de travail courant ;
- Table de références des fichiers ouverts.

Tous les processus sont dotés d'un numéro de processus parent, puisqu'ils proviennent tous d'un autre processus

Sauf pour le processus originel : **init** dont le PID = 1.

En fait, au moment du démarrage du système un pseudo-processus (PID=0) crée un processus **init** dont le PID=1 et le PPID=0. Après ce pseudo-processus disparaît...

Si vous ajoutez le **&** à la fin de la ligne de commande, vous reviendrez directement au prompt du Shell

La commande **jobs** permet de s'informer des processus en cours de traitement en tache de fond

Passage Arrière-plan <-> Premier plan

Un processus en cours d'exécution au premier plan doit être suspendu, à l'aide de la combinaison de touches **Ctrl + Z** .

Une fois le processus au premier plan suspendu, on peut l'envoyer à l'arrière plan avec la commande **bg** (*background*)

Il pourra alors continuer son exécution.

La commande **bg** attend comme paramètre un numéro de tâche précédé d'un signe de pourcentage (%).

Exemple :

\$ test.sh > Sortie.txt

Ctrl + Z

[1]+ Stopped test.sh > Sortie.txt &

\$ jobs -l

[1]+ 1162 Arrêté test.sh > Sortie.txt

\$ bg %1

[1]+ test.sh > Sortie.txt &

\$ jobs -l

[1]+ 1162 Running test.sh > Sortie.txt &

Si vous souhaitez passer un processus d'arrière plan en avant plan, il faut utiliser la commande **fg**, qui attend comme paramètre un numéro de tâche précédé du signe %.

Exemple :

\$ jobs -l

[1]+ 1162 Running test.sh > Sortie.txt &

\$ fg %1

test.sh > Sortie.txt

Ctrl + C pour arrêter ce processus.

Suspendre puis reprendre un processus en arrière-plan

Un processus au premier plan peut être suspendu avec **Ctrl + Z** si l'on veut ensuite le placer en arrière plan.

Pour un processus en arrière plan que l'on veut suspendre puis redémarrer, il faut utiliser la commande **kill**.

Exemple :

On lance un processus en arrière plan :

```
$ test.sh > Sortie.txt &
```

```
[1] 1165
```

On liste les jobs :

```
$ jobs -l
```

```
[1]+ 1165 Running  test.sh > Sortie.txt &
```

On interrompe ce processus :

```
$ kill -STOP %1
```

```
$ jobs -l
```

```
[1]+ 1165 Signal d'arrêt      test.sh > Sortie.txt &
```

On relance ce processus :

```
$ kill -CONT %1
```

```
$ jobs -l
```

```
[1]+ 1165 Running  test.sh > Sortie.txt &
```

Affichage des processus

ps (*Process Status*), nous informe des processus en cours de traitement

\$ ps -l

F	S	UID	PID	PPID	C	PRI	NI	ADDR	SZ	WCHAN	TTY	TIME	CMD
000	S	500	948	672	0	70	0	-	641	wait4	pts/2	00:00:00	bash
000	R	500	1169	948	0	79	0	-	772	-	pts/2	00:00:00	ps

TIME La durée de traitement (temps de processeur) du processus

WCHAN Le canal d'attente (*waiting channel*), par lequel se fait la coordination avec les autres processus

SZ La taille totale du programme en mémoire (SIZE)

Arrêter un processus

Pour arrêter un processus, il faut utiliser la commande **kill**. Elle permet d'envoyer un signal à un process

Syntaxe : **kill [-Numéro-du-signal] PID**

La liste des signaux :

\$ kill -l

Les principaux signaux :

1 (SIGHUP) : Permet aux démons Unix de relire leur fichier de configuration et se reconfigure selon le contenu du nouveau fichier.

- **syslogd** relit le fichier **/etc/syslog.conf**
- **inetd** relit le fichier **/etc/inetd.conf**
- **sendmail** relit le fichier **/etc/sendmail.cf**
- **named** relit le fichier **/etc/named.boot (.conf)**

2 (SIGINT) : Signal d'interruption d'un process. Envoyé par la combinaison de touches Ctrl + C.

9 (SIGKILL) : Tue le processus sans lui demander son avis.

15 (SIGTERM) : C'est le signal par défaut de la commande kill. C'est une requête aimable à un processus de se terminer proprement